

15-440 Distributed Systems

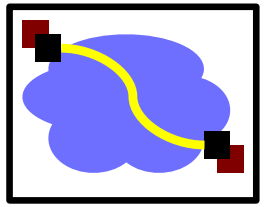
Lecture 20
Virtual Machines
Tuesday Nov 16th, 2015

Some slides based on material from:

Ken Birman @ Cornell, CS6410, Eyal DeLara @ Utoronto, ECE1799

JP Singh, Princeton, COS 318, Alex Snoeren @ UCSD (CSE 120),
OS Concepts (Sliberschatz, Galvin, Gagne 2013)

Logistics Updates

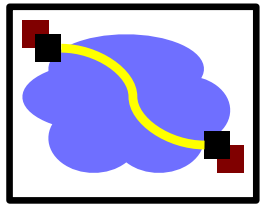


- P2 due Nov 15th.
- HW3 due today in class (11/10)

- HW4 released Today
  - Due Dec 6th
  - Remember: Homework scored Best 3 out of 4
- P3 released Wednesday Nov 16th, DueDec 9th
  - Recitation Nov 21st by TA's => go over P3 writeup
  - Teams => let TA's know if project team change from P2

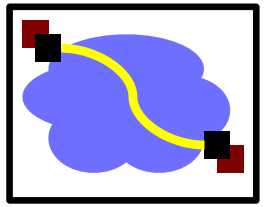
- No office hours for Yuvraj Nov 17th (travelling)

Virtualization



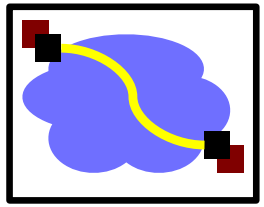
- *"a technique for hiding the physical characteristics of computing resources from the way in which other systems, applications, or end users interact with those resources. This includes making a single physical resource appear to function as multiple logical resources; or it can include making multiple physical resources appear as a single logical resource"*

The idea of Virtualization: from 1960's



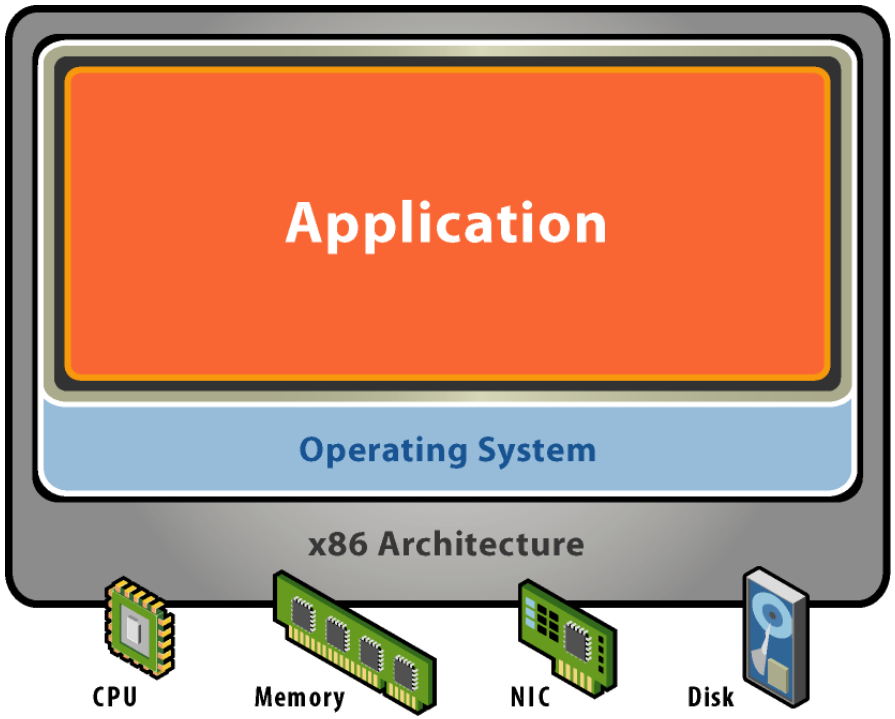
- IBM VM/370 – A VMM for IBM mainframe
  - Multiple OS environments on expensive hardware
  - Desirable when few machine around
- Popular research idea in 1960s and 1970s
  - Entire conferences on virtual machine monitors
  - Hardware/VMM/OS designed together
  - Allowed multiple users to share a batch oriented system
- Interest died out in the 1980s and 1990s
  - Hardware got more cheaper
  - Operating systems got more powerful (e.g. multi-user)

A Return to Virtual Machines



- Disco: Stanford research project (SOSP ’97)
  - Run commodity OSes on scalable multiprocessors
  - Focus on high-end: NUMA, MIPS, IRIX
- Commercial virtual machines for x86 architecture
  - VMware Workstation (now EMC) (1999-)
  - Connectix VirtualPC (now Microsoft)
- Research virtual machines for x86 architecture
  - Xen (SOSP ’03)
  - plex86
- OS-level virtualization
  - FreeBSD Jails, User-mode-linux, UMLinux

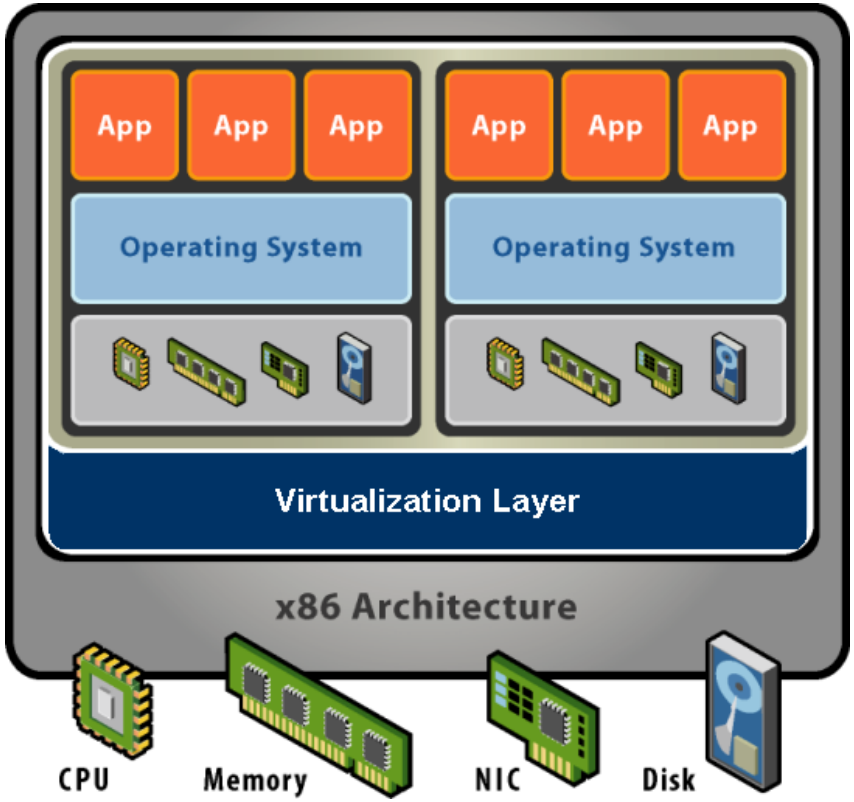
Starting Point: A Physical Machine



- Physical Hardware
  - Processors, memory, chipset, I/O devices, etc.
  - Resources often grossly underutilized
- Software
  - Tightly coupled to physical hardware
  - Single active OS instance
  - OS controls hardware

Adapted from: Eyal DeLara

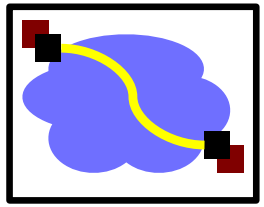
What is a Virtual Machine?



- Software Abstraction
  - Behaves like hardware
  - Encapsulates all OS and application state
- Virtualization Layer
  - Extra level of indirection
  - Decouples hardware, OS
  - Enforces isolation
  - Multiplexes physical hardware across VMs

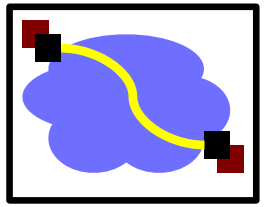
Adapted from: Eyal DeLara

Virtualization Properties, Features

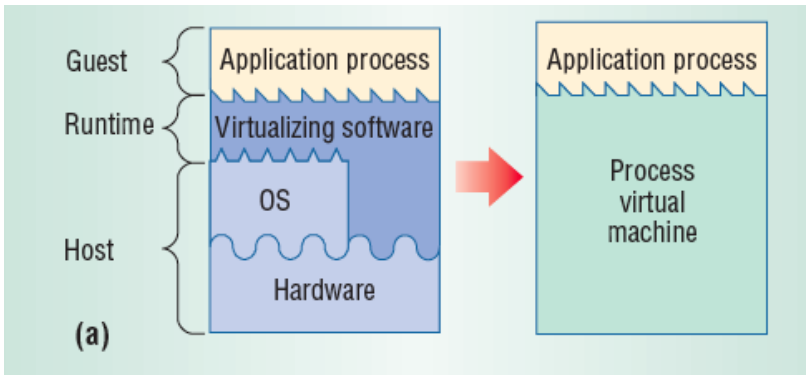


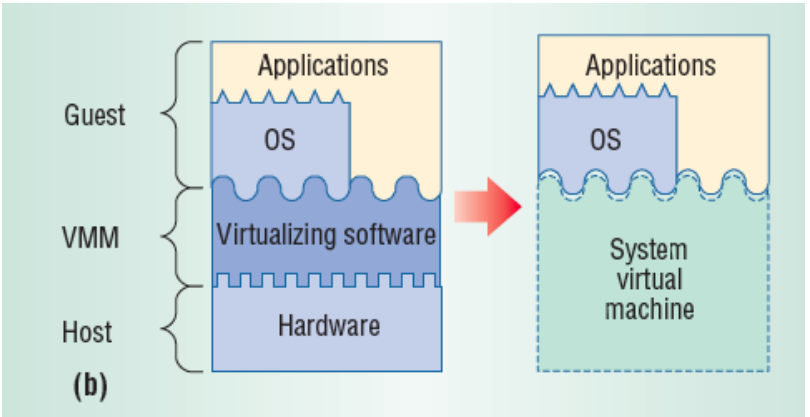
- Isolation
  - Fault isolation
  - Performance isolation (+ software isolation, …)
- Encapsulation
  - Cleanly capture all VM state
  - Enables VM snapshots, clones
- Portability
  - Independent of physical hardware
  - Enables migration of live, running VMs (freeze, suspend,…)
  - Clone VMs easily, make copies
- Interposition
  - Transformations on instructions, memory, I/O
  - Enables transparent resource overcommitment, encryption, compression, replication …

Types of Virtualization



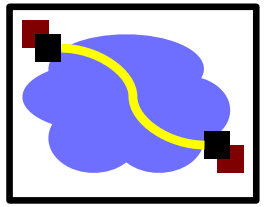
- Process Virtualization (Figure [a])
  - Language-level Java, .NET, Smalltalk
  - OS-level processes, Solaris Zones, BSD Jails, Docker Containers
  - Cross-ISA emulation Apple 68K-PPC-x86
- **System Virtualization** (Figure [b])
  - VMware Workstation, Microsoft VPC, Parallels
  - VMware ESX, Xen, Microsoft Hyper-V





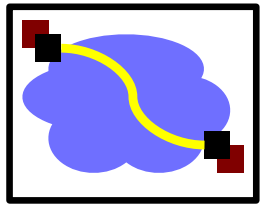
Adapted from: Eyal DeLara

Language Level Virtualization



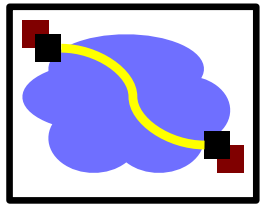
- not-really-virtualization but using same techniques, providing similar features
  - Programming language is designed to run within custom-built virtualized environment (e.g. Oracle JVM)
- Virtualization is defined as providing APIs that define a set of features made available to a language and programs written in that language to provide an improved execution environment
  - JVM compiled to run on many systems
  - Programs written in Java run in the JVM no matter the underlying system
  - Similar to **interpreted languages**

Types of VMs – Emulation



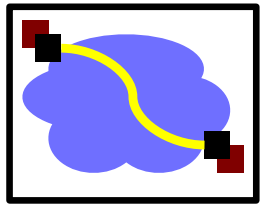
- Another (older) way for running one OS on a different OS
  - Virtualization requires underlying CPU to be same as guest was compiled for while Emulation allows guest to run on different CPU
- Need to translate all guest instructions from guest CPU to native CPU
  - Emulation, not virtualization
- Useful when host and guest have differnet processor architectures
  - Company replacing outdated servers with new servers containing different CPU architecture, but still want to run old applications
- Performance challenge – order of magnitude slower than native code
  - New machines faster than older machines so can reduce slowdown
- Where do you think it is used still?
- Very popular – especially in gaming where old consoles emulated on new

VMs – Application Containers

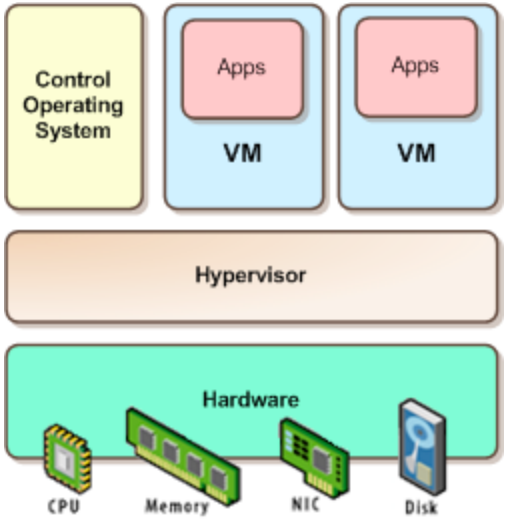


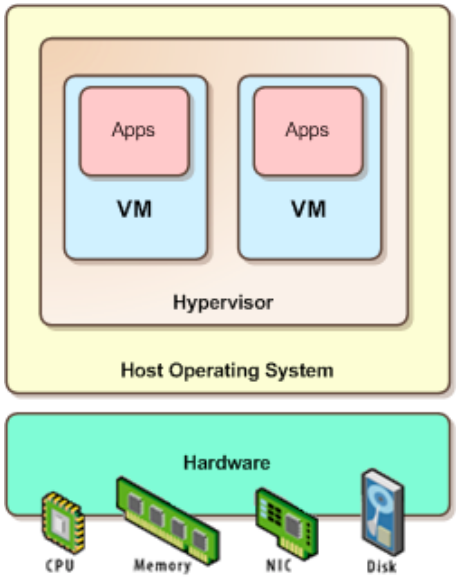
- Some goals of virtualization are segregation of apps, performance and resource management, easy start, stop, move, and management of them
- Can do those things without full-fledged virtualization
  - If applications compiled for the host operating system, don't need full virtualization to meet these goals
- Oracle **containers** / **zones** for example create virtual layer between OS and apps
  - Only one kernel running – host OS
  - OS and devices are virtualized, providing resources within zone with impression that they are only processes on system
  - Each zone has its own applications; networking stack, addresses, and ports; user accounts, etc
  - CPU and memory resources divided between zones
    - Zone can have its own scheduler to use those resources

Types of System Virtualization



- Native/Bare metal (Type 1)
  - Higher performance
  - ESX, Xen, HyperV
- Hosted (Type 2)
  - Easier to install
  - Leverage host's device drivers
  - VMware Workstation, Parallels

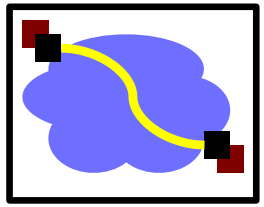




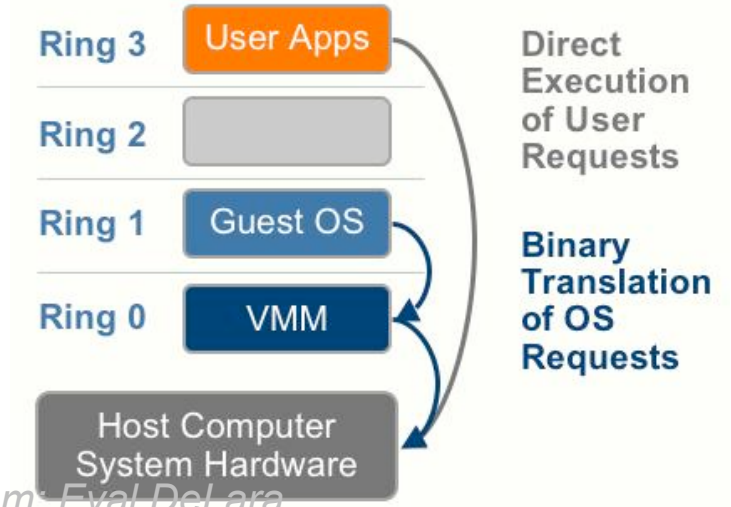
Adapted from: Eyal DeLara

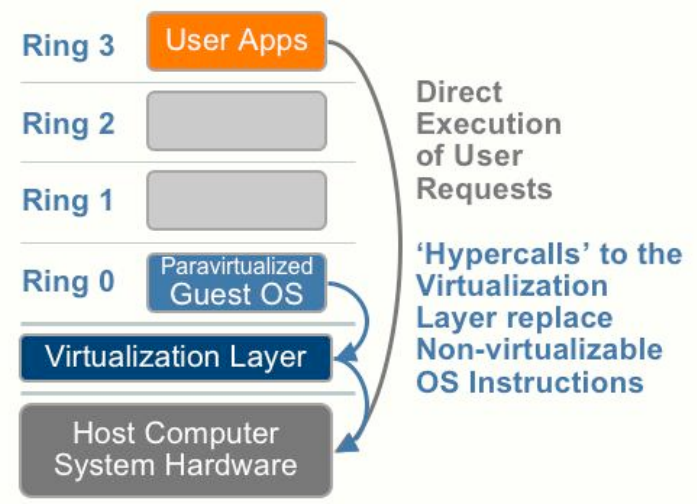
Attribution: http://itechthoughts.wordpress.com/tag/full-virtualization/

Types of Virtualization



- Full virtualization (e.g. VMWare ESX)
  - Unmodified OS, virtualization is transparent to OS
  - VM looks exactly like a physical machine
- Para virtualization (e.g. XEN)
  - OS modified to be virtualized,
  - Better performance at cost of transparency

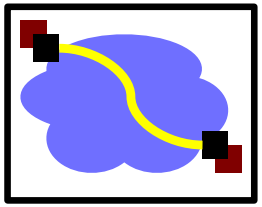




Adapted from: Eyal DeLara

Attribution http://forums.techarena.in/guides-tutorials/1104460.htm

What is a Virtual Machine Monitor?

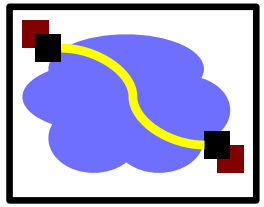


- Classic Definition (Popek and Goldberg '74)

> A virtual machine is taken to be an *efficient, isolated duplicate* of the real machine. We explain these notions through the idea of a *virtual machine monitor* (VMM). See Figure 1. As a piece of software a VMM has three essential characteristics. First, the VMM provides an environment for programs which is essentially identical with the original machine; second, programs run in this environment show at worst only minor decreases in speed; and last, the VMM is in complete control of system resources.

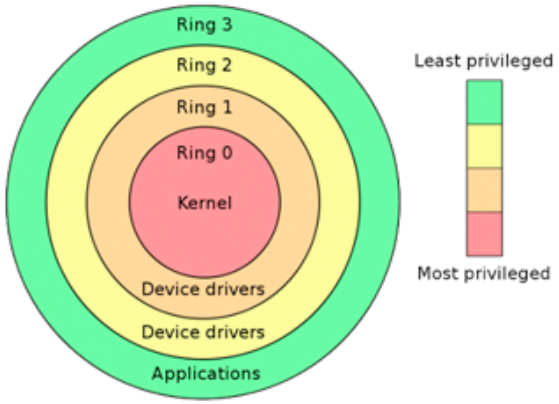
- VMM Properties
  - **Equivalent execution:** Programs running in the virtualized environment run identically to running natively.
  - **Performance:** A statistically dominant subset of the instructions must be executed directly on the CPU.
  - **Safety and isolation:** A VMM most completely control access to system resources.

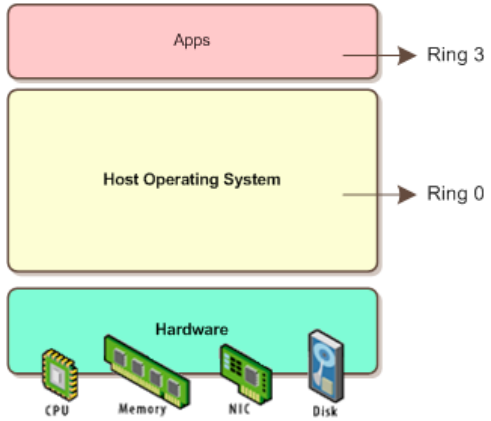
VMM Implementation Goals

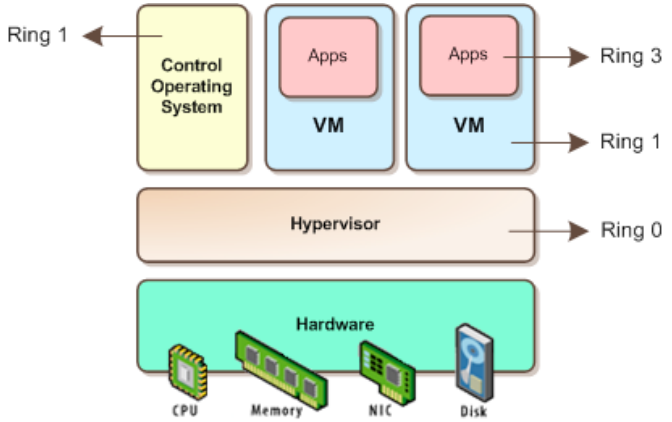


- Should efficiently virtualize the hardware
  - Provide illusion of multiple machines
  - Retain control of the physical machine
- Which subsystems should be virtualized?
  - Processor => Processor Virtualization
  - Memory => Memory Virtualization
  - I/O Devices => I/O virtualization

Processor Virtualization







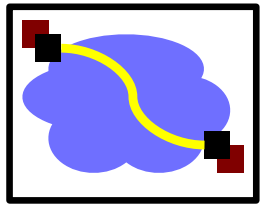
An architecture is classically/strictly virtualizable if all its sensitive instructions (those that violate safety and encapsulation) are a subset of the privileged instructions.

- **all instructions either trap or execute identically**
- **instructions that access privileged state trap**

Attribution: http://itechthoughts.wordpress.com/tag/full-virtualization/

Adapted from: Eyal DeLara

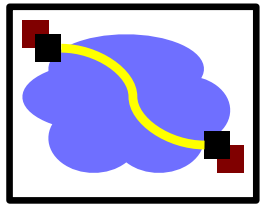
System Call Example



Process	Operating System	VMM
1.System call: Trap to OS		
		2. Process trapped: call OS trap handler (at reduced privilege)
	3. OS trap handler: Decode trap and execute syscall; When done: issue return-frrom-trap	
		4. OS tried to return from trap; do real return-from-trap
5. Resume execution (@PC after trap)		

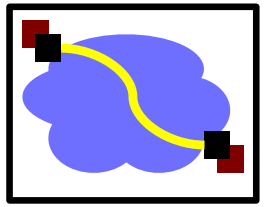
- *Run guest operating system deprivileged*
- *All privileged instructions trap into VMM*
- *VMM emulates instructions against virtual state*
  e.g. disable virtual interrupts, not physical interrupts
- *Resume direct execution from next guest instruction*

X86 Virtualization challenges



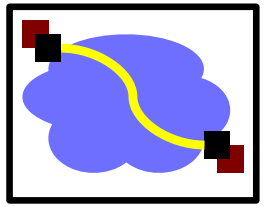
- X86 not Classically Virtualizable
  - x86 ISA has instructions that RD/WR privileged state
  - …But which don't trap in unprivileged mode
- Example: POPF instruction
  - Pop top-of-stack into EFLAGS register
  - EFLAGS.IF bit privileged (interrupt enable flag)
  - POPF silently ignores attempts to alter EFLAGS.IF in unprivileged mode! => no trap to return control to VMM
- Techniques to address inability to virtualize x86
  - Replace non-virtualizable instructions with easily Virtualized ones statically (Paravirtualization)
  - Perform Binary Translation (Full Virtualization)

Virtualizing the CPU



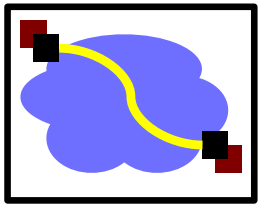
- The VMM still need to multiplex VMs on CPUs
- How could this be done?
  - # Physical CPUs more than #Virtual CPUs?
  - # Virtual CPUs more than #Physical CPUs?
- Timeslice the VMs, each VM runs OS/Apps
- Use simple CPU scheduler
  - Round robin, work-conserving (give extra to other VM)
  - Can oversubscribe and give more #VCPUs that actual

Virtualizing Memory

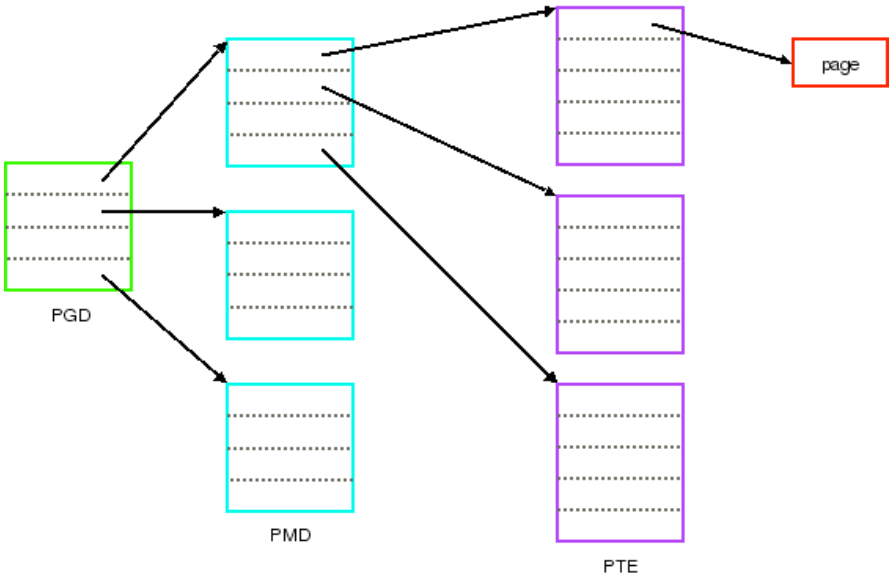


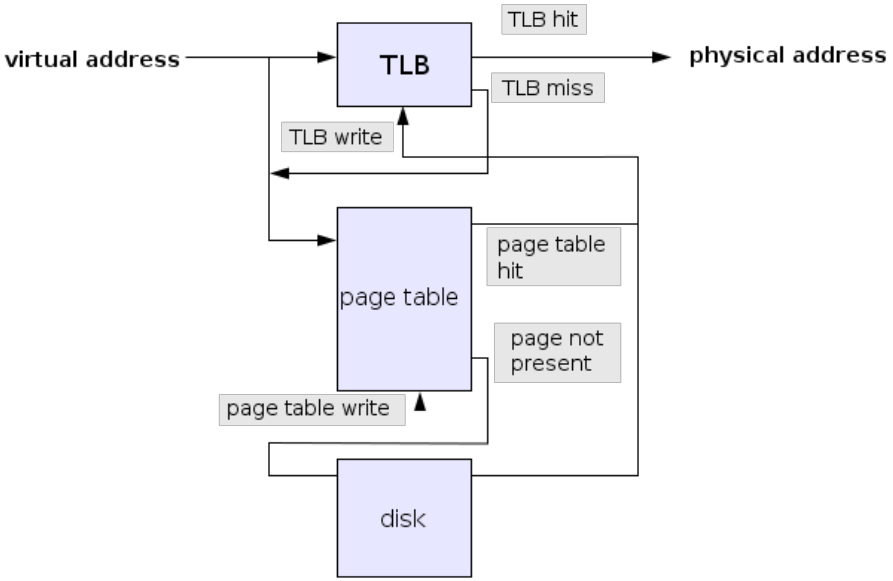
- OS assumes that it has full control over memory
  - Management: Assumes it owns it all
  - Mapping: Assumes it can map any Virtual-> Physical
- However, VMM partitions memory among VMs
  - VMM needs to assign hardware pages to VMs
  - VMM needs to control mapping for isolation
    - Cannot allow OS to map any Virtual => hardware page
- Hardware-managed TLBs make this difficult
  - On TLB misses, the hardware walks page tables in mem
  - VMM needs to control access by OS to page tables

x86 Memory Management Primer

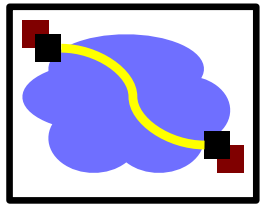


- The processor operates with virtual addresses
- Physical memory operates with physical addresses
- x86 includes a hardware translation lookaside bufer (TLB)
  - Maps virtual to physical page addresses
- x86 handles TLB misses in HW
  - HW walks the page tables => Inserts virtual to physical mapping



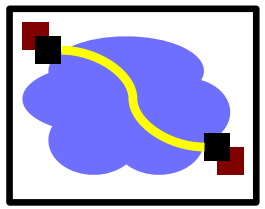


Shadow Page Tables

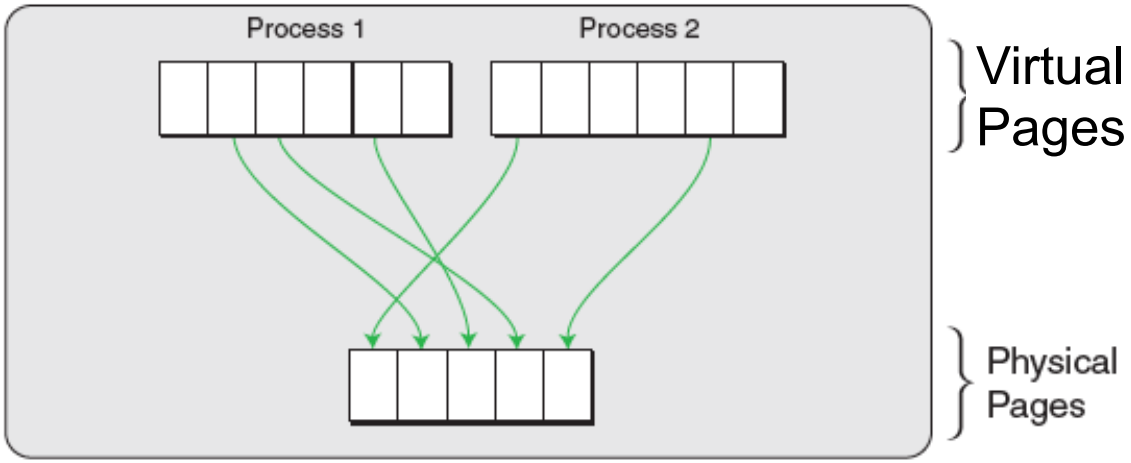


- Three abstractions of memory
  - **Machine:** actual hardware memory (e.g. 2GB of DRAM)
  - **Physical:** abstraction of hardware memory, OS managed
    - E.g. VMM allocates 512 MB to a VM, the OS thinks the computer has 512 MB of contiguous physical memory
    - (Underlying machine memory may be discontiguous)
  - **Virtual:** virtual address space
    - Standard 2^32 address space
- In each VM, OS creates and manages page tables for its virtual address spaces without modification
  - But these page tables **are not used** by the MMU

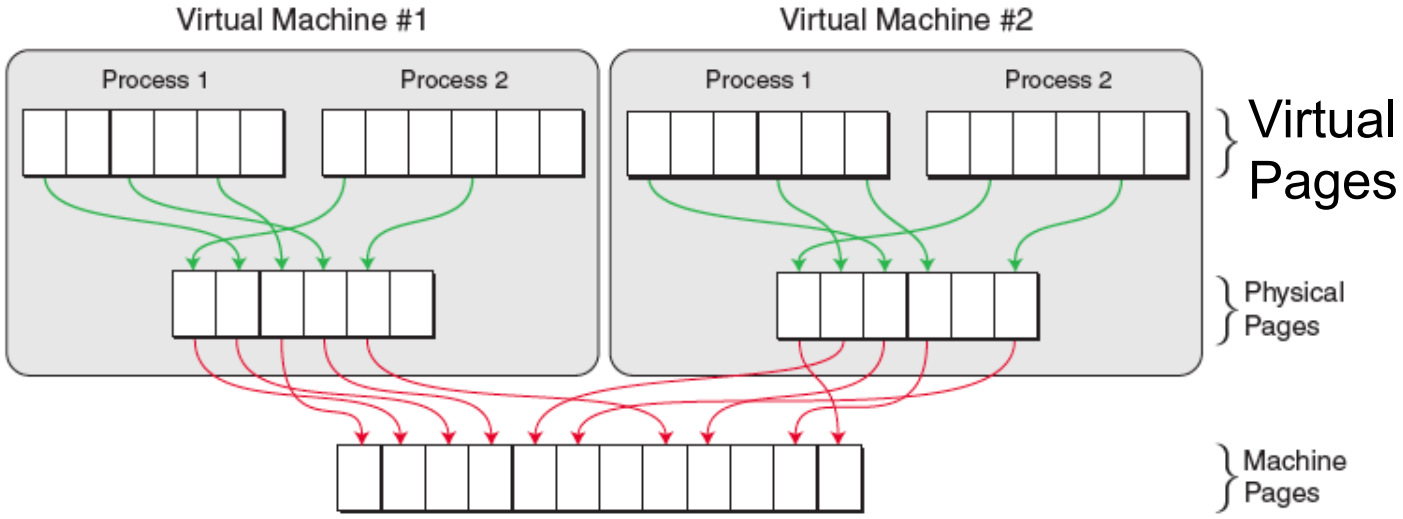
Three Abstractions of memory



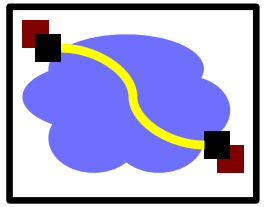
- Native



- Virtualized

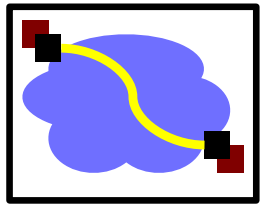


Shadow Page Table (continued)



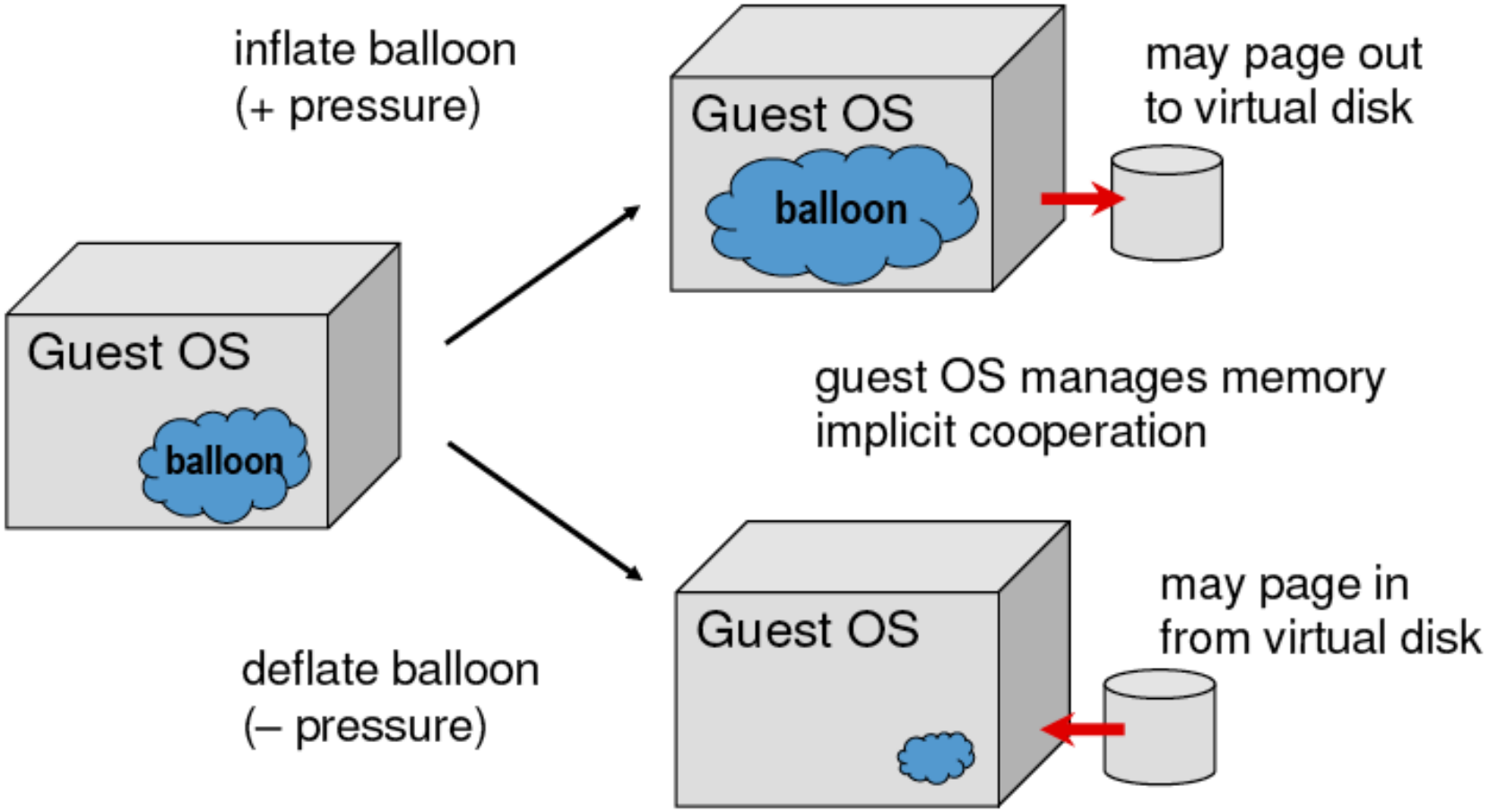
- VMM creates and manages page tables that map **virtual pages** directly to **machine pages**
  - These tables are loaded into the MMU on a context switch
  - VMM page tables are the shadow page tables
- VMM needs to keep its V => M tables consistent with changes made by OS to its V=>P tables
  - VMM maps OS page tables as read only
  - When OS writes to page tables, trap to VMM
  - VMM applies write to shadow table and OS table, returns
  - Also known as memory tracing
  - Again, more overhead...

Memory Management / Allocation

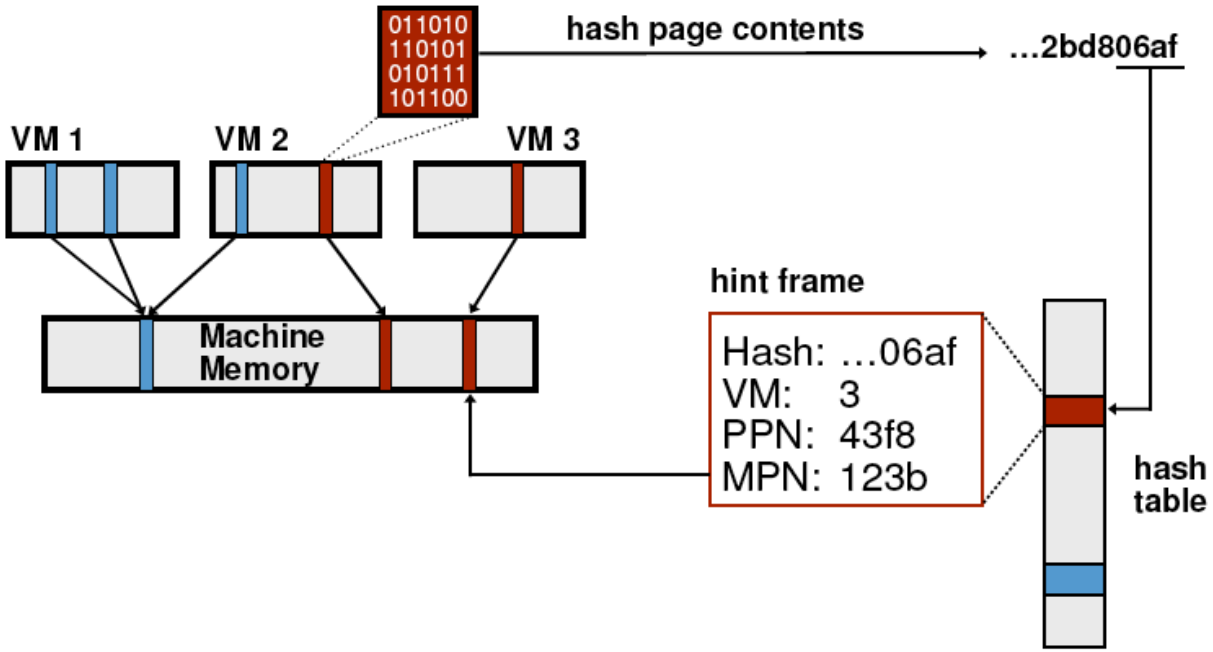


- VMMs tend to have simple memory management
  - Static policy: VM gets 8GB at start
    - Dynamic adjustment is hard since OS cannot handle
    - No swapping to disk
- More sophistication: Overcommit with balooning
  - Baloon driver runs inside OS => consume hardware pages
  - Baloon grows or shrinks (gives back mem to other VMs)
- Even more sophistication: memory de-duplication
  - Identify pages that are shared across VMs!

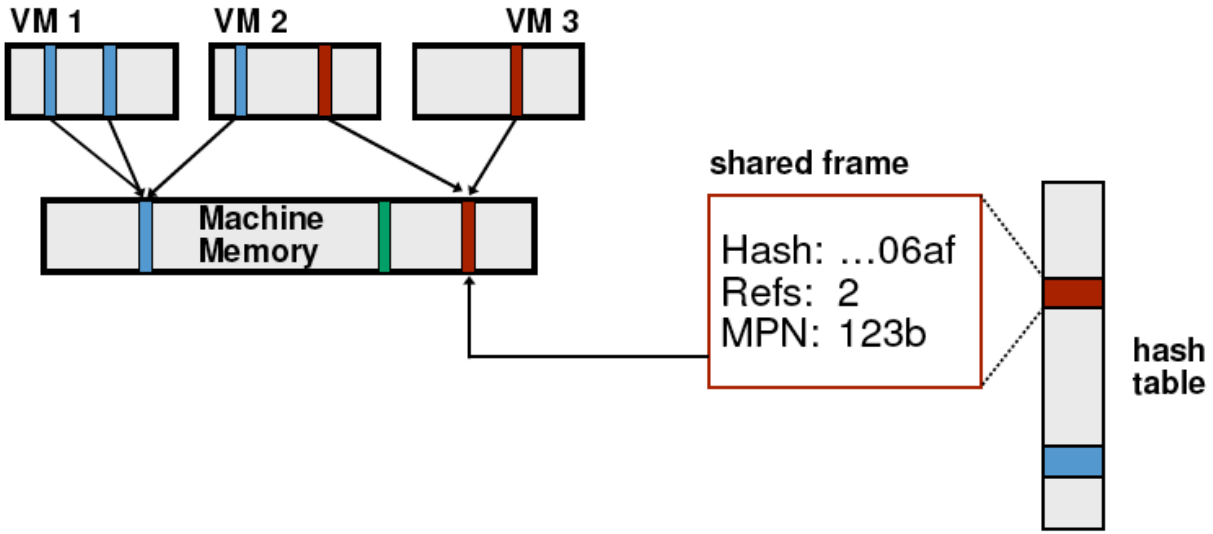
Memory Ballooning



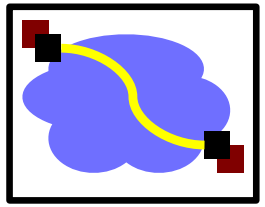
Page Sharing



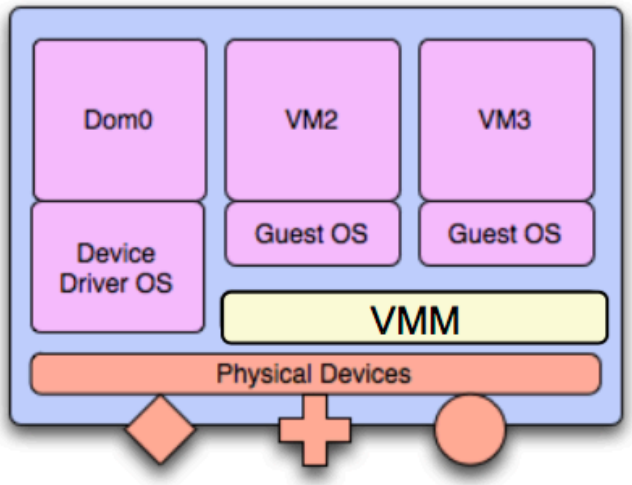
Page Sharing

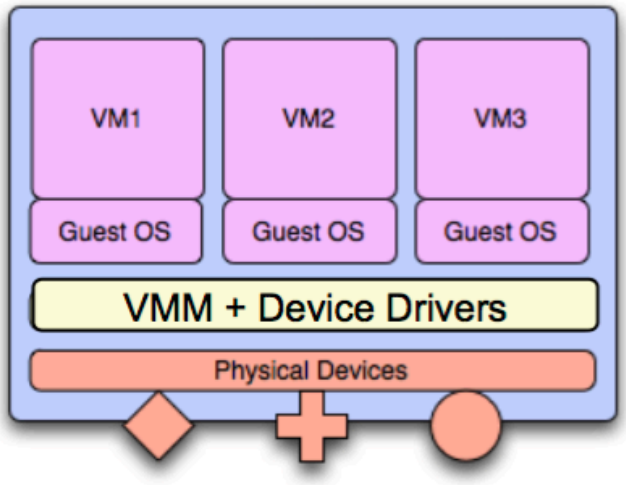


I/O Virtualization

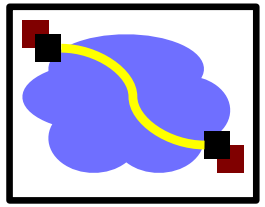


- Challenge: Lots of I/O devices
  - Problem: Writing device drivers for all I/O devices in the VMM layer is not a feasible option
- Insight: Device driver already written for popular Operating Systems
  - Solution: Present *virtual* I/O devices to *guest* VMs and channel I/O requests to a trusted *host* VM (popular OS)



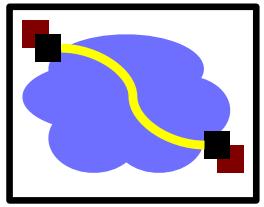


Virtualizing I/O devices



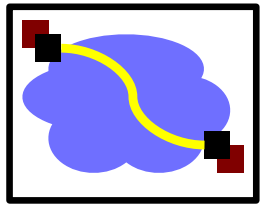
- However, overall I/O is complicated for VMMs
  - Many short paths for I/O in OSes for performance
  - Better if hypervisor needs to do less for I/O for guests,
  - Possibilities include direct device access, DMA pass-through, direct interrupt delivery (need H/W support!)
- Networking also complex as VMM and guests all need network access
  - VMM can **bridge** guest to network (direct access)
  - VMM can provide **network address translation** (**NAT**)
    - NAT address local to machine on which guest is running, VMM provides address translation to guest to hide its address

VM Storage Management



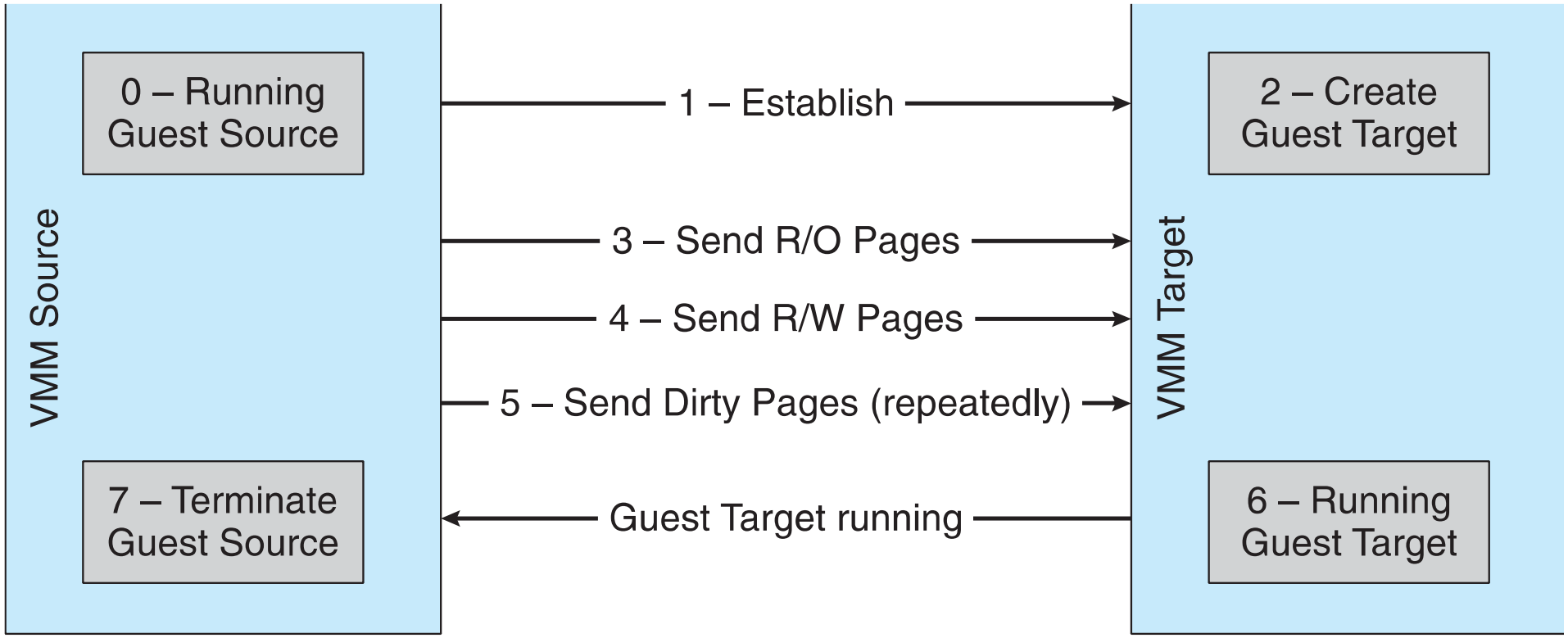
- VMM provides both boot disk + other storage
  - Type 1 VMM – storage guest root disks and config information within file system provided by VMM as a **disk image**
  - Type 2 VMM – store as files in the host file system/OS
- Example of supported operations:
  - Duplicate file (clone) -> create new guest
  - Move file to another system -> move guest
  - Convert formats: **Physical-to-virtual** (**P-to-V**) and **Virtual-to-physical** (**V-to-P**)
- VMM also usually provides access to network attached storage (just networking) => live migration

OS Component – Live Migration

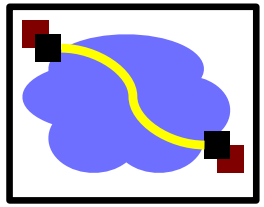


- VMMs allow new functionality like live migration
  - Running guest OS can be moved between systems, without interrupting user access to the guest or its apps
  - Very useful for resource management, no downtime windows, etc
- How does it work?
  1. Source VMM connects to the target VMM
  2. Target VMM creates a new guest (e.g. create a new VCPU, etc)
  3. Source sends all read-only guest memory pages to the target
  4. Source sends all RD/WR pages to the target, marking them clean
  5. Source repeats step 4, as some pages may be modified => dirty
  6. When cycle of steps 4 and 5 becomes very short, source VMM freezes guest, sends VCPU's final state + other state details, sends final dirty pages, and tells target to start running the guest
  7. Target acknowledges that guest running => source terminates guest

Live Migration of VMs



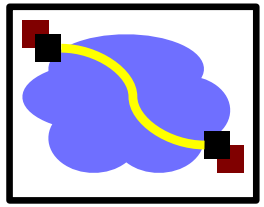
Example VMM: XEN : Introduction



- A Para-Virtualized Interface
- Can host multiple and different OSes
- Supports Isolation
- Performance Overhead is minimum
- Can Host up to 100 Virtual Machines

- Trivia: started at Cambridge, sold for a lot of $$
- Open Source software, xen.org at the moment

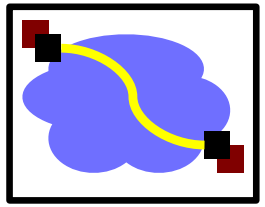
XEN : Approach



- Drawbacks of Full Virtualization with respect to x86 architecture
  - Support for virtualization not inherent in x86 architecture
  - Certain privileged instructions did not trap to the VMM
  - Virtualizing the MMU efficiently was difficult
  - Other than x86 architecture deficiencies, it is sometimes required to view the real and virtual resources from the guest OS point of view

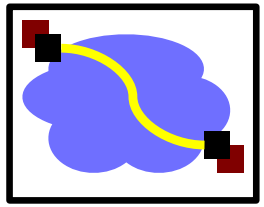
- Xen's Answer to the Full Virtualization problem:
  - It presents a virtual machine abstraction that is similar but not identical to the underlying hardware -para-virtualization
  - Requires Modifications to the Guest Operating System
  - No changes are required to the Application Binary Interface (ABI)

Terminology Used



- **Guest Operating System** (OS) – refers to one of the operating systems that can be hosted by XEN.
- **Domain** – refers to a VM within which a Guest OS runs with applications on top of the OS.
- **Hypervisor** – XEN (VMM) itself.
- Guest OS's (domU) and priviledge domain "dom0"

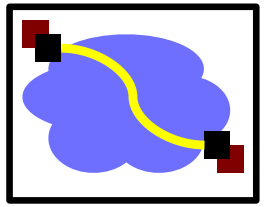
XEN's VMI : CPU



- Problems
  - Inserting the Hypervisor below the Guest OS means that the Hypervisor will be the most privileged entity in the whole setup
  - If the Hypervisor is the most privileged entity then the Guest OS has to be modified to execute in a lower privilege level
  - Exceptions

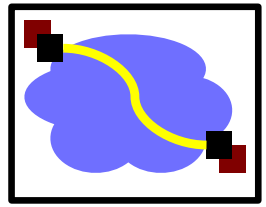
- Solutions
  - x86 supports 4 distinct privilege levels – rings
  - Ring 0 is the most and Ring 3 is the least
  - Allowing the guest OS to execute in ring 1- provides a way to catch the
  - privileged instructions of the guest OS at the Hypervisor
  - Exceptions such as memory faults and software traps are solved by registering the handlers with the Hypervisor
  - Guest OS must register a fast handler for system calls with the Hypervisor
  - Each guest OS will have their own timer interface

XEN's VMI: Device I/O



- Existing hardware Devices are not emulated
- A simple set of device abstractions are used – to ensure protection and isolation
- Data is transferred to and fro using shared memory, asynchronous buffer descriptor rings – performance is better
- Hardware interrupts are notified via a event delivery mechanism to the respective domains

XEN : Cost of Porting Guest OS



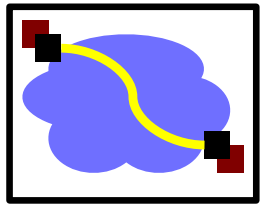
- Linux is completely portable on the Hypervisor - the OS is called XenoLinux
- Lot of modifications to the architecture specific code was done in both the Oses
- In comparing both OSes – Larger Porting effort for XP

OS subsection	# lines	
	Linux	XP
Architecture-independent	78	1299
Virtual network driver	484	–
Virtual block-device driver	1070	–
Xen-specific (non-driver)	1363	3321
Total	**2995**	**4620**
(Portion of total x86 code base	**1.36%**	**0.04%)**

Table 2: The simplicity of porting commodity OSes to Xen. The cost metric is the number of lines of reasonably commented and formatted code which are modified or added compared with the original x86 code base (excluding device drivers).

Adapted from: Ken Birman

XEN : Control and Management



- Xen exercises just basic control operations such as access control, CPU scheduling between domains etc.
- All the policy and control decisions with respect to Xen are undertaken by management software running on one of the domains – domain0
- The software supports creation and deletion of VBD, VIF, domains, routing rules etc.

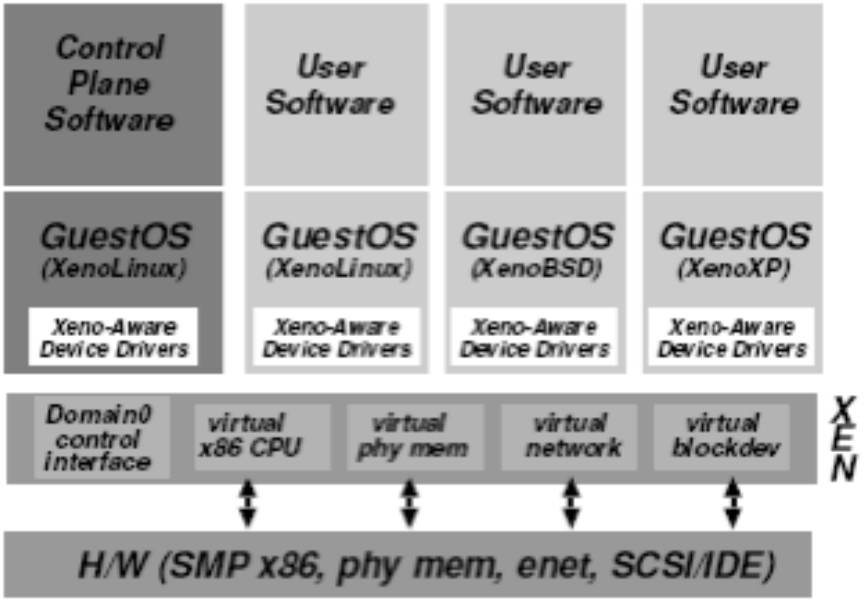
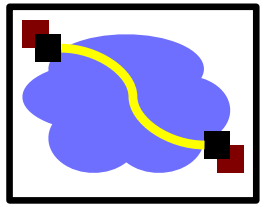


Figure 1: The structure of a machine running the Xen hypervisor, hosting a number of different guest operating systems, including *Domain0* running control software in a XenoLinux environment.

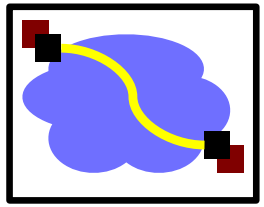
Adapted from: Ken Birman

XEN : Disk Management



- Only Domain0 has direct unchecked access to the physical disks.
- Other Domains access the physical disks through virtual block devices (VBDs) which is maintained by domain0.
  - VBS comprises a list of associated ownership and access control information, and is accessed via I/O ring.
- A translation table is maintained for each VBD by the hypervisor, the entries in the VBD's are controlled by domain0.
- Xen services batches of requests from competing domains in a simple round-robin fashion.

Summary



- Introduction to Virtualization: why do it?
- Different types of Virtualization: Container based, Language Based, Type I/II, Paravirtualized.
- VMM techniques to virtualize the CPU, memory, I/O devices, etc
- Example: XEN, open source VMM